Глава 12. Многопоточность. RTOS

Ежелев Г. И.

11 мая 2026 г.

Управление задачами

- Весь код, который вы писали до этого - выполнялся последовательно. То есть был какой-то основной цикл в котором выполнялся весь код.
- Такой подход называется Super loop
  - Такой код быстрее писать и удобнее понимать
  - Завис один узел - гарантированно зависла вся программа
  - Сложно писать распределение нагрузки. Если регулятор нужно обновлять раз в 100 мс, раз в 10 мс нужно опрашивать датчик и раз в 30 мс отправлять пакеты экрану - на каждое действие должен быть свой таймер (а если частота важна - то прерывание).
  - Многозадачность реализуется конечным автоматом.

Конечный автомат

Конечный автомат

Конечный автомат (Finite State Machine, FSM) - это программа, имеющая конечное число состояний и правил перехода между ними. В целом довольно простая мысль. Суть в том, как эти состояния реализовать.

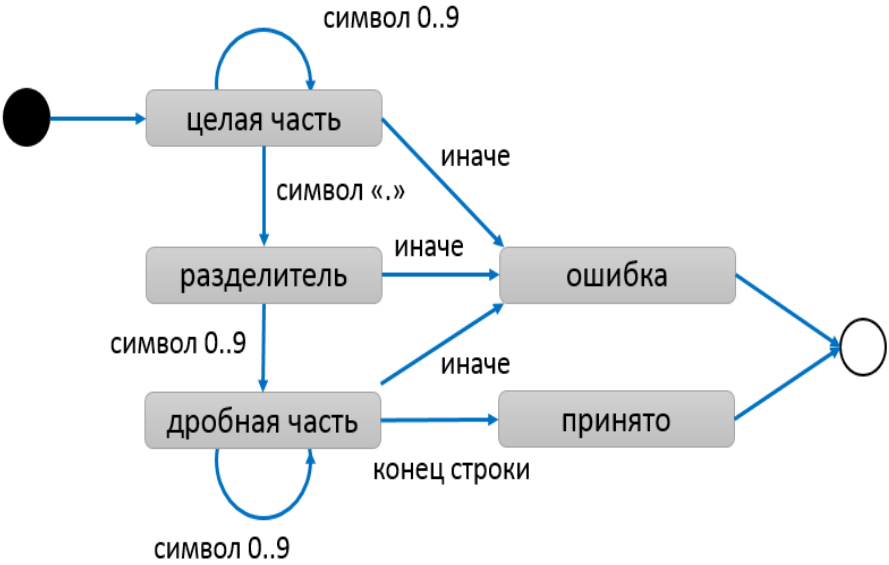
В случае робототехники:

- Цикл делится на 3 части: Listen-Think-Act (Слушай-Думай-Делай).
- Listen - первая часть (в самом начале), занимается обработкой датчиков (никаких чтений датчиков в других частях цикла - это костыль)
- Think - Конечный автомат. Он обычно делится на две части.
  • Первая - только выбор состояния на основании данных с датчиков. То есть набор условий:
  if(state == 1) [условия перехода из 1 -> {2,..., n}]
  ....
  if(state == n)[условия перехода из 1 -> {1,..., n-1}]

• Вторая часть - только выполнение состояний (без переходов между ними):
if(state == 1) [Действие в состоянии 1]
....
if(state == n)[Действие в состоянии n] • Такой подход позволяет не запутаться во всех состояниях, когда код разрастеться.

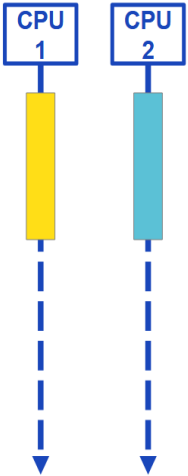
- Act - В конце - отдача приказов узлам робота (моторы, экранчик и т.п.)никаких моторов в середине кода!.
- Listen-Think-Act - это пример паттерна проектирования - это некоторый шаблон или идея, как надо реализовывать какой-то кусок кода (т.к. не вы первые догадались датчики читать и потом по данным с них двигаться).
- Многие паттерны звучат очевидно и скучно, но при этом почему-то люди всё равно их игнорируют и пишут плохой код ☺

Пример. Парсинг строки



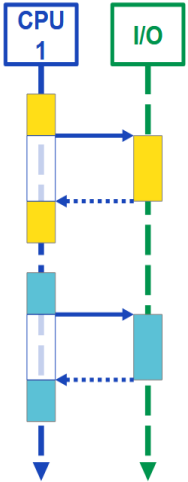
Управление задачами

- Два процессора
- Две задачи
- Параллельное исполнение
- Более сложное управление



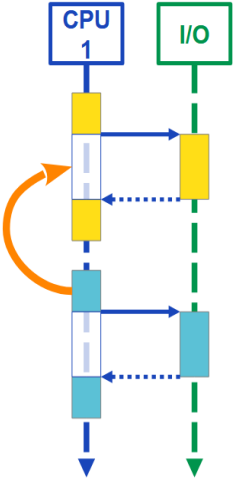
Управление задачами

- Один процессор
- Две задачи
- Ввод-вывод
- Процессор простаивает



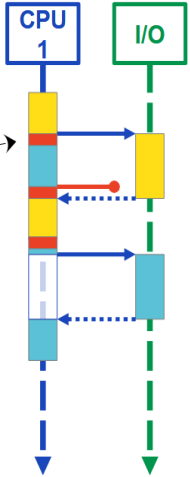
Управление задачами

- Один процессор
- Две задачи
- Ввод-вывод
- Процессор простаивает
  и это можно использовать



Управление задачами

- Один процессор
- Две задачи
- Ввод-вывод
- Переключение контекста
- Конкурентное исполнение
- Более сложное управление



Конкурентность

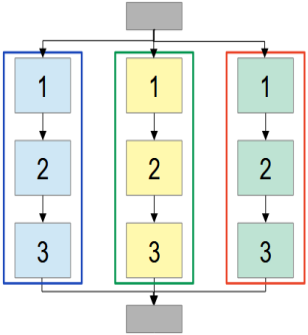
- Конкурентность - исполнение нескольких задач одновременно. Это не всегда значит, что задачи физических работают одновременно. Главное - они реализованы, как отдельные независимые задачи, которые не знают о существовании других.
- В случае, если одна задача отдает на время управление, программа переключается на исполнение другой задачи. В этот момент происходит переключение контекста - сохранение данных одной задачи и подгрузка данных следующей.
- Даже с одним ядром - такой подход имеет свои плюсы - вам не нужно думать, как распределять ресурсы между задачами (моторы раз в 100 мс, регуляторы раз в 10...). Каждая идейная задача - отдельная параллельная задача в коде, система сама переключит контекст когда надо.

Итог

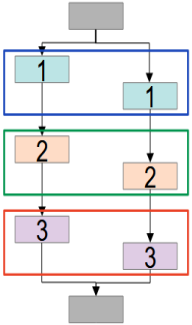
- Один процессор
- Много задач
- Многозадачность
- Конкурентное исполнение

Варианты архитектур

- распараллеливание
  - обработчик выполняет все этапы одной задачи



- конвейерная обработка
  - обработчик выполняет один этап всех задач



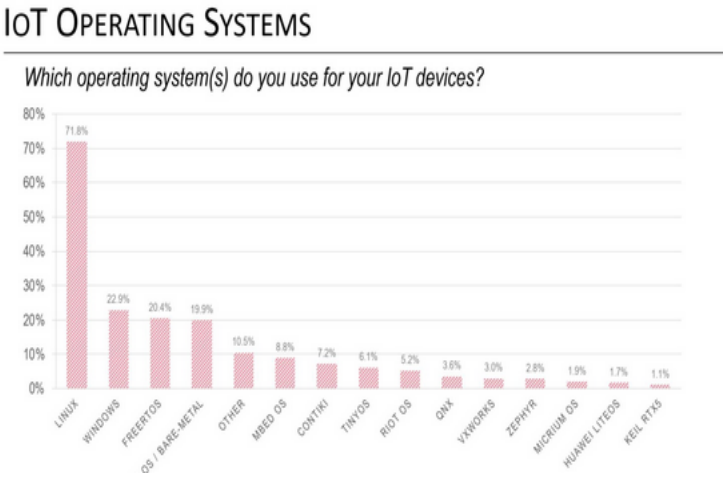
RTOS

Все манипуляции с процессами (например переключение контекста) осуществляет планировщик(Scheduler) - часть операционной системы.
•Операционные системы реального времени (Real Time Operating System, RTOS) - это ОС, которые ставят в приоритет не равномерность распределения ресурсов между задачами и удобство пользователя, а временную детерминированность, т.е. реакция на событие произойдет в строго(почти) определенное время.
Пример такой системы - FREERTOS, состоящей из:

- Планировщика(scheduler) - решает, какая задача выполняется сейчас
- Диспетчера задач (Task manager) - следит за состоянием задачи, памятью и тп.
- Управление памятью (Heap manager) - управление данными задач (их целых 4-5 штук)
- Механизмы синхронизации - очереди, семафоры, уведомления, группы событий.

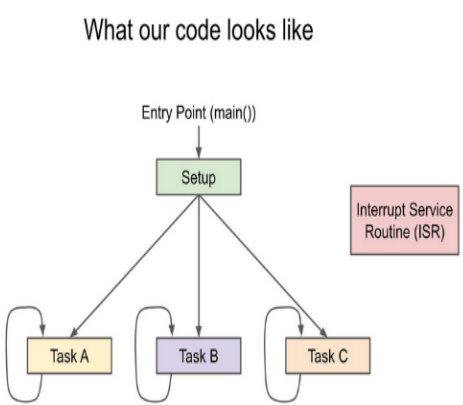
Популярность RTOS

По опросу Eclipse Foundation для IoT (Internet of Things) устройств после Linux и Windows- самой популярной основой стал FREERTOS, обогнав разработку без операционной системы (Bare Metal)

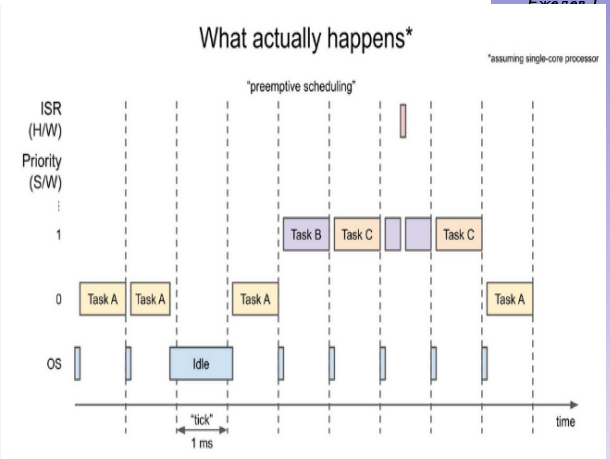


Задачи во FREERTOS

Задача - это какая-то функция, обычно реализованная как (почти)бесконечный цикл, планировщик их сам нарезает:



Как мы видим задачи в коде



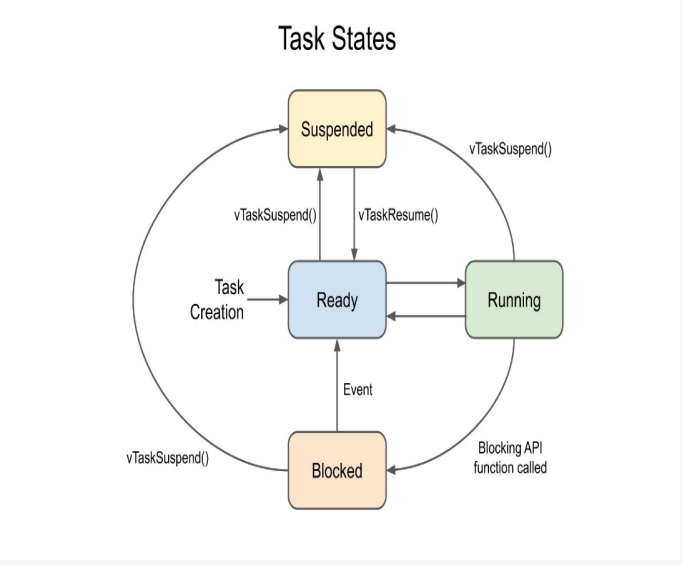
Их реальное исполнение

Переключение задач

- Каждую миллисекунду FreeRtos пробуждает ядро по тому самому прерыванию SysTick. Каждый период между пробуждениями ОС так и называется - "тик".
- Задачи переключаются операционкой в следующих ситуациях:
- По приоритету задач. В новый тик есть заждавшаяся задача того же приоритета или любая задача более высокого приоритета. Это реализовано через массив очередей(у каждого уровня приоритетов своя очередь) - pxReadyTasksLists[configMAX_PRIORITIES].
- Вытеснение (Preemption). Если неважная задача А ставиться на паузу прерыванием, например приемом пакета UART (а прерывания никто не отменял), а после приема пакета в очередь попадает более важная задача Б, то RTOS сразу последует в Б, не завершая А.

- Ожидание. Мы можем поставить задачу в ожидание, пока какая-то другая задача что-то не сделает, что пробудит нашу задачу.
- Блокировка. Задачи могут обмениваться данными. Если задача попробует запросить данные у другой, которых еще нет, она заблокируется, пока данные не появятся, а в это время будут исполняться другие таски

Состояния задач

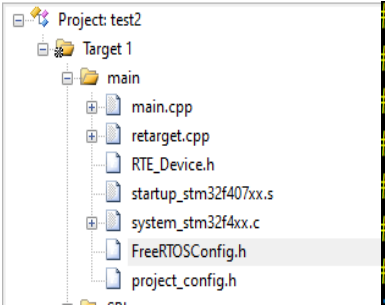


Состояния задач

- Ready - После создания задача попадает в это состояние. Такие задачи готовы к исполнению и ждут внимания ядра
- Running - выполнение
- Blocked - заблокирована. Например вызвана задержка, или задача попыталась запросить данные от другой задачи, а их нет (Блокировка) или задача ожидает событие (Ожидание). В этом состоянии задача лежит в планировщике.
- Suspended - приостановлена. Более радикальная мера, задача выбрасывается из планировщика. Нужна для отключения какого-то куска логики. Зачем нужно? => чтобы сохранить контекст и данные самой задачи.

сборка проекта FREERTOS

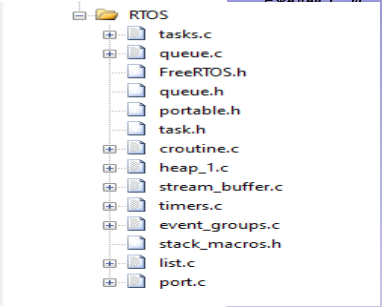
Все файлы операционки уже есть в проекте. проверьте что все нужные файлы добавлены в keil:



FreeRtosConfig.h

```
#pragma once
#include "misc.h"
#include "project_config.h"
#include "FreeRTOS.h"
#include "task.h"
#include "queue.h"
#include "portable.h"
```

Не забудьте подключить



Все файлы операционки

Закоментировать SysTick в нашем Time.h, FREERTOS его тоже использует для синхронизации

Советую посмотреть файл FreeRtosConfig.h (он выложен в репе, скопируйте к себе, в проекте он возможно неправильный)

Создание задачи

```
BaseType_t xTaskCreate(
    TaskFunction_t pvTaskCode,
    const char * const pcName,
    configSTACK_DEPTH_TYPE usStackDepth,
    void *pvParameters,
    UBaseType_t uxPriority,
    TaskHandle_t *pxCreatedTask
);
```

- pcName - имя задачи (просто строка)
- usStackDepth - глубина стека задачи (об этом позже)
- pvParameters - объект для передачи параметров если ничего не передаем - NULL
- uxPriority - приоритет задачи для планировщика - больше ⇔ важнее
- pxCreatedTask - структура для управления задачей извне, можно передать NULL
- Отметим, что в FREERTOS есть одна всегда запущенная задача с низшим приоритетом - Idle Task (простой) (она даже есть выше на диаграмме what actually happens).
- Нельзя использовать RETURN!
- А как переключать задачи?

Запуск задачи

В файле main.cpp:

```
74
75  xTaskCreate(f1, "blinker", configMINIMAL_STACK_SIZE, NULL, tskIDLE_PRIORITY+1, NULL);
76  xTaskCreate(f2, "blinker2", configMINIMAL_STACK_SIZE, NULL, tskIDLE_PRIORITY+1, NULL);
77  vTaskStartScheduler();
```

```
void f1(void *pvParams)
{
  while(true)
  {
    //your code...
  }
}
```

main.cpp

Пример. Светодиод + аналоговый сигнал. Код на github

```
1  #pragma once
2  #include "misc.h"
3  #include "project_config.h"
4  #include "FreeRTOS.h"
5  #include "task.h"
6  #include "queue.h"
7  #include "portable.h"
8  int state = 0;
9
10 // Function prototypes
11 void vTaskLED(void *pvParameter
12 void vTaskADC(void *pvParameters);
13
14 volatile uint16_t ADCValue;
15
```

- Импорт библиотек
- Далее прототипы функций
- ADCValue - где будем хранить данные с ADC. Не заубдьте про модификатор volatile

Пример. Запуск и инициализация. Код на github

```
16 int main(void) {
17     // Peripheral init
18     /*RCC_Config();
19       GPIO_Config();
20       ADC_Config();*/
21
22     //
23     xTaskCreate(vTaskLED, "LED_Task", configMINIMAL_STACK_SIZE, NULL, tskIDLE_PRIORITY+1, NULL);
24     xTaskCreate(vTaskADC, "ADC_Task", configMINIMAL_STACK_SIZE, NULL, tskIDLE_PRIORITY+1, NULL);
25
26     // Start FreeRTOS scheduling
27     vTaskStartScheduler();
28
29     while(1); // program won't reach this line
30 }
31
```

Пример. Задача светодиода. Код на github

```
32 void vTaskLED(void *pvParameters) {
33     while(true) {
34         //-------alternate gpio signal---------
35         // gpio.write(a);
36         // a = abs(a - 1);
37         //-----------------
38
39         //delay for blinking
40         vTaskDelay(pdMS_TO_TICKS(500));
41     }
42 }
43
```

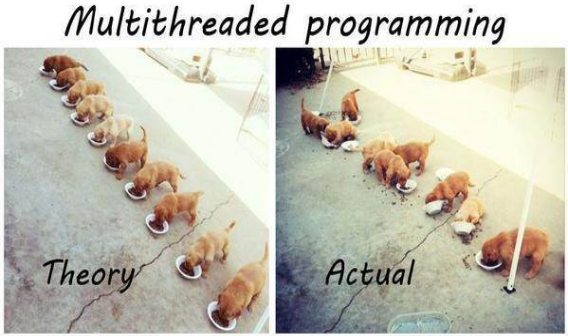
- Изменение сигнала светодиода 1->0 или 0->1
- Задержка для мигания 500 тиков, т.е. 500мс

Пример. Задача чтения аналогового сигнала Код на github

```
44 void vTaskADC(void *pvParameters) {
45     while(1) {
46
47         // Waiting new ADC data -> can be added as a part of READ function to ADC class
48         while(ADC_GetFlagStatus(ADC1, ADC_FLAG_EOC) == RESET);
49
50         // reading new value
51         ADCValue = ADC_GetConversionValue(ADC1);// or = ADC1->DR
52
53         //
54         vTaskDelay(pdMS_TO_TICKS(10));
55     }
56 }
```

Синхронизации

- Сейчас мы запускали задачи, которые работали независимо друг от друга. В реальных проектах - это редкость.
- Можно выделить 3 основных вида взаимодействия задач:
  - Обмен данными
  - Ожидание других задач
  - Ограничения доступа к ресурсам



Обмен данными. Уведомления

```
1  //-----NOTIFICATIONS--------
2  TaskHandle_t xHandler;
3
4  xTaskNotifyGive(xHandler);
5
6  ulTaskNotifyTake(pdTRUE, portMAX_DELAY);
7  //--------------------/
```

- xHandler - как раз структура для контроля задачи извне (последний аргумент при запуске задачи).
- xTaskNotifyTake - ожидание уведомление. Если уведомлений нет - задача засыпает. В случае pdTrue - при выходе из TaskNotifyTake счетчик уведомлений обнуляется и задача проходит один раз. В случае pdFALSE - счетчик уменьшается на 1.
- xTaskNotifyGive - уведомить задачу по её TaskHandler, т.е. увеличить счётчик уведомлений на 1.

- Для счетчика уведомления используют поле в структуре TCB (Task Control Block) - специальный блок, который выделяется после создания задачи.
- В нем лежат uint32_t ulNotifiedValue - ячейка с любым числом и uint8_t ucNotifyState - байт состояния (ждем уведомления; не ждем; получили, но еще не прочитали).
- Их и использует механизм уведомлений. Теоретически - в ячейку ulNotifiedValue можно положить любые данные, что позволяет существенно сэкономить память, не создавая очереди и т.п.

Обмен данными. StreamBuffer. Отправка. Пример в github

- Stream Buffer - это блокирующий способ обмена информацией между задачами. Все данные хранятся побайтово.
- Если в нем закончится место - он остановит задачу, которая попытается в него что-то положить, пока не пройдет TicksToWait или место не освободиться
- Очень удобен для работы с IO, например в случае UART у нас в любом случае приходят и отправляются байты, поэтому такой примитивной и легковесной структурынам хватит

Обмен данными. StreamBuffer. Отправка. Пример в github

```
size_t xStreamBufferSend(
    StreamBufferHandle_t xStreamBuffer,
    const void *pvTxData,
    size_t xDataLengthBytes,
    TickType_t xTicksToWait
);
```

- xStreamBuffer - хэндлер буфера, структура аналогичная хэндлеру для задачи, т.е. а.к.а. отссылка на конкретный буффер в памяти
- *pvTxDara - указатель на начало фрагмента памяти с данными
- xDataLengthBytes - длина фрагмента памяти в байтах
- xTicksToWait - тайм-аут если в буфере закончилось место (Защита от вечной блокировки).

Обмен данными. StreamBuffe

```
size_t xStreamBufferReceive(
    StreamBufferHandle_t xStreamBuffer,
    void *pvRxData,
    size_t xBufferLengthBytes,
    TickType_t xTicksToWait
);
```

аналогичная хэндлеру для задачи, т.е. а.к.а. отссылка на конкретный буффер в памяти

- *pvRxDara - указатель на начало фрагмента памяти для чтения
- xDataLengthBytes - таймаут блокировки, если буфер пустой (нечего читать).
- xTicksToWait - тайм-аут если в буфере закончилось место (Защита от вечной блокировки).
- textcolorredTrigger Level - необязательный параметр, который устанавливает минимальное количество байт в буфере чтобы проснуться (например для общения по UART это нужно, чтобы вы могли ожидать прием всего пакета, а не нескольких байт из него) (Trigger level есть в примере на github).

Обмен данными. Queue.Приём. Пример в github

Следующий метод это очередь. Работает схожим образом, но по умолчанию хранит структуры в одной ячейке

```
BaseType_t xQueueSend(
    QueueHandle_t xQueue,
    const void * pvItemToQueue,
    TickType_t xTicksToWait
);
```

```
BaseType_t xQueueReceive(
    QueueHandle_t xQueue,
    void * pvBuffer,
    TickType_t xTicksToWait
);
```

- xQueue - хэндлер очереди, структура аналогичная хэндлеру для задачи, т.е. а.к.а. отссылка на конкретную очередь в памяти
- *pvItemQueue - указатель на объект куда/откуда писать
- xTicksToWait - тайм-аут если в буфере закончилось место (Защита от вечной блокировки).

Ожидание задач. Семафоры. Пример в github

- Семафор - это счётчик разрешений. Можно делать бинарными (1 разрешение), можно множественными.
- Задача берет разрешение, если оно есть, то счётчик уменьшается на 1, иначе зависает, в ожидании, что другая задача отдаст разрешение (в этом случае счетчик увеличиться на 1).
- При неудаче (нет доступных разрешений и прошел тайм-аут возвращает какое-то значение, можете его погуглить)

Ожидание задач. Виды семафоров. Пример в github

- Отличаются только тем, что вы проинициализируете для первого аргумента (на гитхабе пример бинарного семафора)
- Бинарный семафор - одно разрешение
- Счётный семафор - внутри счётчик разрешений
- Mutex - Mutual Exclusion. Нужен для разграничения доступа к ресурсам. Это бинарный семафор с важным условием - возвращать его может только тот, кто его взял. Это концепция владения ресурсом. Нельзя использовать в прерываниях ⇒ используйте бинарный семафор.

```
BaseType_t xSemaphoreTake(
    SemaphoreHandle_t xSemaphore,
    TickType_t xTicksToWait
);
```

```
BaseType_t xSemaphoreGive(
    SemaphoreHandle_t xSemaphore
);
```

Группы событий. Пример в github.

- EventGroups - механизм для ожидания множественных флагов
- Какие-то задачи выставляют флаги какие-то флаги, сигнализируя события (например блок работы с камерой сигнализирует прием новых данных, а блок управления кикером сигнализирует, что удар перезарядился).
- Другие задачи могут засыпать в ожидании определенных комбинаций битов или хотя бы одного

Группы событий. Пример в github.

```
EventBits_t xEventGroupSetBits(
    EventGroupHandle_t xEventGroup,
    const EventBits_t uxBitsToSet
);
```

```
EventBits_t xEventGroupWaitBits(
    EventGroupHandle_t xEventGroup,
    const EventBits_t uxBitsToWaitFor,
    const BaseType_t xClearOnExit,
    const BaseType_t xWaitForAllBits,
    TickType_t xTicksToWait
);
```

- xEventGroup - хэндлер группы.
- uxBitsToSet - какие биты выставить.
- uxBitsWaitFor - Какие биты ждать.
- xClearOnExit - При срабатывании группы: pdTrue (аналог true для FreeRtos) $\Longrightarrow$ сбросить биты, иначе (pdFalse) не сбрасывать.
- xWaitForAllBits - pdTrue - ждать все биты (логическое И), иначе ждать хотя бы один (логическое или).
- xTicksToWait - тайм-аут.

Взаимодействие задач в прерываниях.

- Прерывание - это не обычная задача.
- Она не учитывается планировщиком, у нее нет стека, хэндера и т.п.
- Блокировка в прерывании вызовет зависание программы (т.к. системных тиков нет)
- Поэтому все вышеописанные механизмы не будут работать в прерываниях
- Для всех структур (кроме mutex -> вместо него бинарный семафор) есть fromISR альтернативы, которые дают не прямую директиву планировщику, а указание, что делать сразу после выхода из прерывания.
- Если всё же внутри прерывания нужен контроль ресурсов, то используются критические секции (мы их уже использовали для UART).
- Везде нужно просто в конце добавлять fromISR, в случае очереди: xQueueSend() $\Longrightarrow$ xQueueSendFromISR()
- Также для каждой функции появляется доп аргумент, который указывает планировщику, нужно ли переключать контекст при